car Honda Accord (with all specified features), and the estimated sale date (say, May 21st, as today is May 7th). The output, on the other hand, is the predicted sale price for this car. In the illustration below, the prediction module predicts a sale price of $11,384:

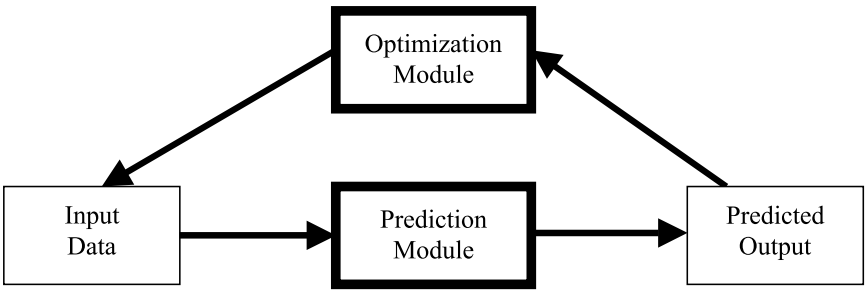
But how does a prediction module "know" what the sale price of a particular Honda Accord will be on a particular auction site at a particular point in time? Well, the prediction module would have to "learn" to predict the sale price based on the historical sales data for all Honda Accords at the auction site in Northern California. For the sake of simplicity, let us assume we have the following Honda Accord sales data from last week's auction in Northern California:

Make / Model	ZIP	Date	Price
KD37D92JF83NF8822	94102	8.11.2004	$9,265
MDK293HFDWH299305	94102	8.11.2004	$12,495
28DN39FNDJW2N0024	94102	8.11.2004	$11,925
29H93NFI3HJF93F04	94102	8.11.2004	$11,396
ND920ENF1NAD02834	94102	8.11.2004	$9,835
D39DJ39EHQ8HH9335	94102	8.11.2004	$8,965
02UFIMF03JF9SH935	94102	8.11.2004	$13,960
D932NF93HG9057362	94102	8.11.2004	$8,830
00F8EB3IDNB293758	94102	8.11.2004	$7,920
IE038THJ203TH0234	94102	8.11.2004	$19,250

Let us also assume that our (very basic) prediction module "predicts" the sale price for each make/model at each auction site by looking at the average sale prices for the previous week (hence, it does not take into account the actual mileage, color, or other variables). It is interesting to note, however, that creating even a basic prediction module such as this presents many difficulties. For instance, a separate prediction model is needed for each make, model, and location. Hence, the prediction module might need to contain around 10,000 separate prediction models (20 makes × 10 models × 50 locations). Furthermore, if our database contains approximately three million cases (data collection spanning 30 months), then each prediction model will have an average of 300 cases. The distribution of these cases will be non-uniform. For example, we might have 2,800 cases per location for some makes/models (e. g., Toyota Camry), but only one or two cases per location for some other makes/models (e. g., Porsche 911)! Consequently, it would be impossible to create a prediction model for some makes/models. Regardless of these complications, let us assume for the moment that our basic prediction module makes a prediction of $11,384 (for the Honda Accord being sent to an auction site in Northern California). Of course, this "prediction" would not be very accurate, but it serves as a good starting point for further discussion in Sect. 4.4 below.

4.3 Optimization

Next comes the development of an optimization module capable of recommending the best answer. Note that the "best answer" is based on the prediction module's output. For example, to evaluate the merit of a particular distribution of cars, we need to predict their sale prices. The relationship between the prediction and optimization modules can be displayed as follows:



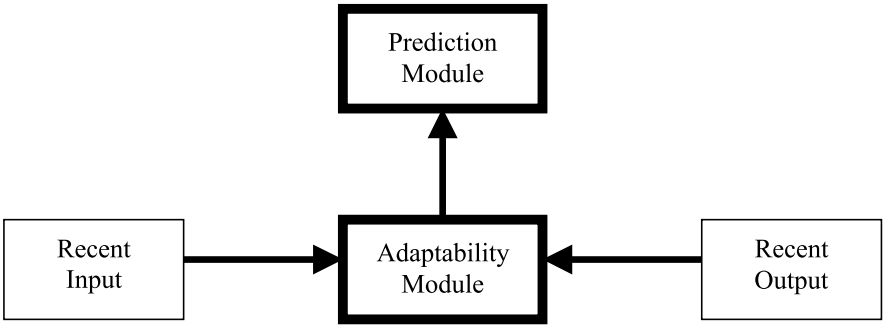
In the above diagram, the optimization module generates a distribution solution that serves as *input data* for the prediction module. This input data provides a destination assignment (i. e., auction site) for each car, which the prediction module uses to generate the predicted sale prices. The optimization module then uses the sum of all the predicted sale prices (i. e., the *output data*) to gauge the quality of the input data: The higher the sum of the predicted car sale prices, the better the input

data. To maximize the sum of all the predicted sale prices, the optimization module tries many different input data combinations and then evaluates the output data (of course, the optimization module must modify the output data for transportation costs and other adjustments). Many different techniques can used to construct an efficient optimization module, which we will discuss in Chap. 6.

4.4 Adaptability

Developing effective prediction and optimization modules is a great start, but by themselves these modules are insufficient for today's ever-changing environments. Because today's accurate prediction might be inaccurate tomorrow, the prediction module must be capable of "learning from" and "adapting to" changes in the environment. The concept of adaptability has far-reaching consequences: Imagine a system that would improve over time by learning from its own prediction errors. Such a system would truly be adaptive!

The adaptability process can be illustrated as follows:



To detect errors between the predicted result and the actual result, an adaptability module compares the predicted sale prices (i. e., recent input) with the actual prices for each car (i. e., recent output). If errors exist, the adaptability module will "tune" the prediction module to decrease the prediction error.

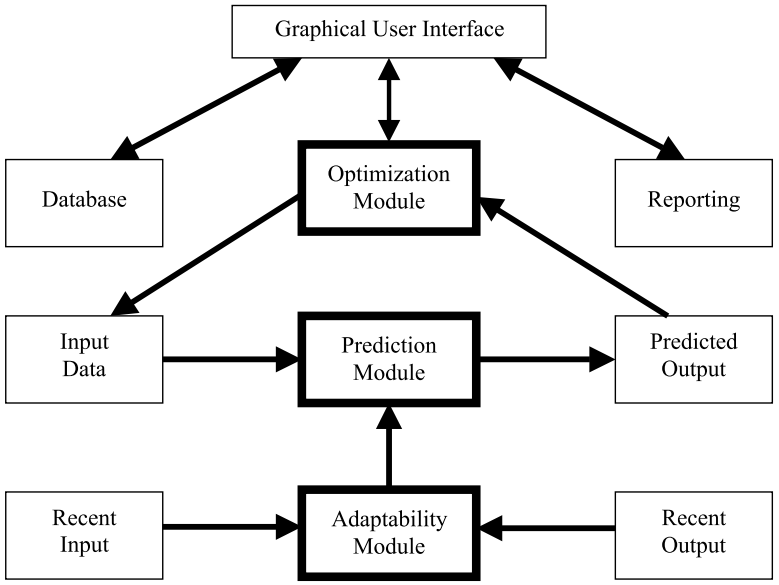
As an example of this process, let us return to the basic prediction module discussed in Sect. 4.2, which "learns" the average sale price for each make/model at each auction site by looking at the sale prices for the previous week. If the adaptability module updates the prediction module every week by using a rolling time window, then the prediction module can adapt to changes in the sale prices. Alternatively, imagine that the prediction module has certain rules that can be expressed as follows:

if [Make = Honda] **and** [Model = Accord] **and** [Color = white] **and** [40,000 < Mileage < 50,000] **and** [Year = 2000] **and** [Damage Level = $0], **then** Sale Price = $11,384.

Each of these rules has a weight,[15] and the weights of rules can be modified (say, on a weekly basis) to tune the predictions in a changing environment. In other words, the adaptability module can "adapt" to environmental changes by updating these rules, with the update frequency depending on how fast the environment changes. Of course, a *really* good adaptability module can make its own decision on the update frequency by continuously measuring its own prediction errors. Hence, a *really* good adaptability module can adapt its own speed of adaptation! It is important to note, however, that the adaptability module's effectiveness in updating the prediction module is influenced by the type of prediction methods that were used to build the underlying models. In the following chapter, we will discuss some of these methods in detail.

4.5 The Structure of an Adaptive Business Intelligence System

The prediction, optimization, and adaptability modules are the core components of an Adaptive Business Intelligence system. However, this does not mean that other components are not important (e. g., an easy-to-use graphical user interface, a database for storing information). Thus, the overall structure of an Adaptive Business Intelligence system resembles the following diagram:



[15] The "weight" of a rule is just a parameter that provides a relative measure of the rule's merit. Rules that are more important are assigned higher values (recall the definition for *parameter* in Sect. 3.1).

In Sect. 2.6, we discussed how prediction, optimization, and adaptability were used in the "pollution control" research project in Poland. The prediction module was responsible for predicting the ecological damage that each 30 km × 30 km square would sustain during the next 48 hours; the optimization module (consisting of an evolutionary algorithm) was responsible for recommending the ideal energy production level for each power station; and the adaptability module was responsible for tuning the system's performance. Furthermore, there were a few reporting and visualization modules incorporated into the system, which interfaced with the databases. In Part III of this book, we will discuss a modern Adaptive Business Intelligence system with all of these components.

Part II:
Prediction and Optimization

5 Prediction Methods and Models

> "When you have eliminated the impossible, whatever remains, *however improbable*, must be the truth."
> *The Sign of Four*

> "As to Holmes, I observed that he sat frequently for half an hour on end, with knitted brows and an abstract air, but he swept the matter away with a wave of his hand when I mentioned it. 'Data! data! data!' he cried impatiently. 'I can't make bricks without clay.' "
> *The Adventure of the Copper Beeches*

Most "prediction problems" can be categorized as classification problems, regression problems, or time series problems. When placing a prediction problem into one of these three categories, two major aspects have to be taken into account: the *expected output* and *time*. Let us explain these two aspects further.

For some problems, there are only two possible expected outputs: "yes" or "no," "true" or "false," "buy" or "sell," etc. These are classic classification problems,[1] because they assign new cases to a class. The best example would be classification of credit card transactions into two classes: "fraudulent" and "legitimate" (this problem is discussed in more detail in Sect. 12.5). A classification problem may have, however, more than two outputs – in fact, the number of possible classes (i. e., expected outputs) might be quite significant (e. g., different types of diseases). In these classification problems, time does not exist; the "future" is understood as an arrival of a new (yet unknown) case, or it is included as a variable of the case.

Similar comments are also applicable to regression problems. The general purpose of (multiple) regression is to discover the relationship between several independent ("predictor") variables and a dependent ("criterion") variable, with the output being a concrete number. For example, we may want to predict salary levels as a function of position, number of years at the position, number of supervised employees, etc. A regression model will also tell us which variables are better predictors than others, and we can easily identify "outliers."[2] Again, the issue of time is either non-existent or included as a variable of the case.

---

[1] A prediction model developed for a classification problem is often called a *classifier*.

[2] An *outlier* is an observation that lies at an abnormal distance from the other values in a random sample. For example, the annual salary level for 1,000 randomly selected people might be in the range of $17,832 to $167,942, with the exception of one person, an outlier, who earns $938,400 per year.

In contrast to classification and regression problems, "time" is the main feature of a time series problem, with each case containing many values measured over some time period in the past. In other words, the time-dependencies among the cases are so strong that the cases must be kept in a sequential time order. In time series problems, the future is referenced explicitly: we would like to predict a variable's value in the future (tomorrow, next month, etc.). A classic example in economics would be to predict next year's Gross Domestic Product (GDP). Plenty of historical data are available (released every quarter), and the prediction model may include many additional economic indicators as variables (e. g., employment, financial, survey, production, and sales indicators).

Despite the fact that prediction problems come in all shapes and sizes – varying in the number of variables, types of data patterns, time horizons, and types of expected output – only two types of prediction methods exist for addressing these problems: *quantitative* and *qualitative* methods. The quantitative methods assume that a sufficient amount of data exists about the past, that these data can be quantified in the form of numerical data, and that past patterns will continue into the future. Conversely, qualitative methods are applied in situations where very little quantitative data are available, but where sufficient qualitative knowledge exists.

Although quantitative methods vary from simple (and intuitive) methods based on empirical experience to formal methods based on statistical principles, all these methods require data! Fortunately, the amount of stored data are growing at a rapid rate. This growth takes place on two dimensions: the number of cases stored (e. g., new transactions) and the number of variables in each case (e. g., the detail of each transaction). In general, the more data the better, as data mining can produce better results when performed on large data sets, and the resulting prediction models are more accurate.

In the car distribution example, there are several important elements of prediction. For example, we would like to predict the sale prices for different cars at different auction sites on different days. Because these predictions are based on past cases, we should know all the variables (e. g., "make," "model," "body style," "mileage") of the cars that were sold over the last, say, three years; and we should also know the sale price, and the exact date and location. Having all this information, we can then apply various prediction methods to develop a good prediction model. Of course, as we discussed in Chap. 3, the prediction model should also take into account the distribution of other cars as well, because of the volume effect.

The process of building a prediction model usually consists of a few steps:

- *Data preparation.* To avoid the situation of "garbage in, garbage out," the relevant data must be "prepared." This step includes data transformation, normalization, creation of derived attributes, variable selection, elimination of noisy data, supplying missing values, and data cleaning. This stage is often augmented by preliminary data analysis to identify the most relevant variables and to determine the complexity of the underlying problem. The data preparation step can be the most laborious, and many people believe that it constitutes 80% of any data mining effort.

- *Model building.* This step includes a complete analysis of the data (i. e., the data mining stage), the selection of the best prediction method on the basis of (a) explaining the variability in question, and (b) producing consistent results, and the development of one or more prediction models.
- *Deployment and evaluation.* This step includes implementing the best prediction model, and applying it to new data to generate predictions. However, because new data arrive on a continuous basis, it is essential to measure the prediction model's performance and tune it accordingly.

Let us examine each of these steps.

5.1 Data Preparation

Generally speaking, there are only two "types" of variables: *numerical* and *nominal*. Numerical variables are numbers (e. g., "34,982" for mileage), while nominal variables take their values from a predefined set (e. g., "black," "white," or "red" for color). Because the values of nominal variables are symbols (strings of characters), there is rarely any order between them, and mathematical comparisons and operations do not make much sense (as it is difficult to add "50" to "blue," or to compare which is larger: "blue" or "green"). Hence, it makes sense to talk about *ordered* nominal variables, where comparisons of the type "greater than," "less than," and "equal to" have meaning.

An additional type of variable is binary (also called a Boolean or true/false variable), as it only takes one of two possible values (e. g., "yes" or "no," "true" or "false"). We may also come across other types of variables (e. g., variables that store free text as a value, or that contain a set of values). Most prediction methods and models require that variables be either binary or numerical (or nominal with numerical codes as values), thus allowing some order. So, what should we do with truly nominal variables, such as color? Well, there are two possibilities: Either the color of a car can be coded as a unique number, or it can be converted into several binary (true/false) variables, with each variable representing a particular color. For example, if the color of the car is white, then the variable can take on the value "true" (or "1"); if the color of the car is not white, then the value would be "false" (or "0").

To properly prepare the data, it is important to first identify the variable "type" (i. e., to know whether the values of a variable allow arithmetical operations or logical comparisons, whether there is a natural order imposed among them, and whether is it meaningful to define a distance between the values). For example, "very light," "light," "medium," "heavy," and "very heavy" follow a natural order, but the distance between them is not defined. Mileage values, on the other hand, have a natural measure of distance: a car with 34,789 miles has 3,500 miles less than a car with 38,289 miles. Because the goal of any prediction model is to produce an output (i. e., the prediction), it is important to note that the output is also a variable. In the car distribution example, the predicted output is the sale price, which is a single numerical variable.

In the data preparation phase, some variables may also require "transformation." For example, it is quite typical for "date of birth" to be recorded as a variable, but many decisions (or queries to the system) may be based on the "age" of an individual. A simple data transformation step would convert the variable "date of birth" into the variable "age" by subtracting a person's date of birth from the current date. Returning to the car distribution example, the VIN is clearly the key variable, because we can use it to identify any car. However, the VIN itself is not useful for data mining activities (after all, the string of 17 characters looks random and meaningless: e. g., JD8320DJ2094GK2X3), but by using a VIN decoder it can be transformed into meaningful information about the make, model, year, and trim-level of a car.

Although data transformation is an important step in the data preparation process, *variable selection* and *variable composition* are even more important. Variable composition – which is somewhat similar to data transformation – requires problem-specific knowledge to create new variables. Because these new variables (often called *synthetic variables*) present existing data in a "better" form, they may have a greater impact on the results than the specific prediction model used to produce these results. A trivial example is the creation of a new variable to record the average miles driven per year, which corresponds to the ratio:

$$\text{Mileage} / (\text{Current Year} - \text{Year} + 1)$$

The denominator would tell us the number of years the car was in service, and the entire ratio would tell us the average miles driven per year.

Variable selection on the other hand (also known as *feature selection* or *attribute selection*) is the process of selecting the most relevant variables. This process should be performed carefully, because if meaningful variables are not selected then everything else – from data transformation all the way to the final prediction model – will be meaningless. Conversely, selecting irrelevant variables may deteriorate the accuracy of a prediction model (in other words, removing irrelevant variables usually improves the performance of a prediction model). This may seem straightforward: After all, there are a finite number of variable subsets, so we can examine all of them and select the best one! Unfortunately, it is not quite that simple. First, the number of possible subsets may be too large: For a database with "only" 20 variables, there are over 1 million possible subsets. Second, to evaluate each subset, we will need to build a prediction model and evaluate it by measuring the prediction error (we will discuss this in detail in Sect. 5.3, along with some other validation issues). So, what is the solution?

Although the best way to select the most relevant variables is still manual (based on problem-specific knowledge), there are numerous automatic methods that can be divided into several categories. For example, we can consider methods that evaluate the relevance of a single variable versus methods that evaluate a subset of variables. Another category of automatic methods is based on the timing of selection: Some methods select variables at the very beginning using characteristics of the data, while other methods select relevant variables during the model construction process.

Let us consider a few (simple) examples of different automatic methods for variable selection that we could apply to a problem. Say the prediction problem is one of classification (e. g., "fraudulent" and "legitimate") and we are trying to evaluate the usefulness of particular variables (such as the time of transaction, or the amount) for predicting the outcome. One of the most popular automatic methods we could use is based on "means and variances." Using a simple statistical test, the means of a variable are compared for the two classes to see whether the difference is likely to be random or not. Small differences in means usually imply irrelevant variables. This method evaluates the variables one by one, and does so before the development of any prediction model. On the other hand, we could use an automatic method where the variable selection process is an inherent part of the prediction model. For example, when a decision tree is built (these are covered in the next section), the relevant variables are selected, one by one, during the tree-building process. Lastly, we could also use automatic methods that evaluate the entire subset of variables. Many optimization techniques discussed in Chap. 6 would be appropriate for this type of approach, as the problem is really an optimization problem (i. e., finding the "optimal" subset of variables).

Because the variable selection step removes redundant and/or non-productive variables, we can consider this step as part of the "data reduction" process, the general goal of which is to delete nonessential data (as the data set may be too big for some prediction models and/or the expected time for building a model might be too long). As data are represented in a table, we can: (1) reduce some variables (columns) in the table, (2) reduce some values present in the table, and/or (3) reduce some cases (rows) from the table. We have already discussed the removal of some variables, which is equivalent to the task of variable selection, so let us move on to reducing values.

It is often necessary to "discretize" a numeric attribute into a smaller number of distinct categories (e. g., the variable "mileage" can be grouped into values of "below 10,000 miles," "between 10,000 and 19,999 miles," "between 20,000 and 29,999 miles," and so on, right up to "over 200,000 miles"). This looks natural, but how can we be sure that such discretization is any good? Moreover, what is a good way to discretize numeric variables into categories? As usual, there are a few possibilities to consider. One approach would be to discretize an attribute by rounding: The actual mileage of the car can be rounded off to the closest 1,000 miles, thus 23,772 miles would become 23,000 miles. Another possibility would be to create some number of discrete categories (say, 20), and distribute all values to these categories in such a way that the average distance of a value from its category mean is the smallest. For example, the first category may contain mileages from 0 to 11,209, the second category may contain mileages from 11,789 to 18,991, and so on. Some mathematical methods (such as *k-means clustering*) can deliver near-optimal solutions for such distributions. However, this approach might be a bit risky for time-changing data. In the case of off-lease cars, new cases are coming in at regular intervals, so the optimal mileage distributions might change quite frequently.

Finally, let us turn our attention to the last possibility of data reduction, which is the reduction of cases from the table. Clearly, the number of cases is often the

largest dimension of the data; it is not unusual to have hundreds of millions of cases containing 20 to 30 variables each. This does not mean, however, that the process of case reduction is easy. Just the opposite is true: very often, case reduction is the hardest type of data reduction to perform. The general approach for handling case reduction is based on random sampling. Rather than using the whole data set to build a prediction model, random samples are used instead. Two popular techniques for random sampling include:

- *Incremental sampling.* Where the model is trained on increasingly larger random subsets of cases, the trends are observed, and the process is stopped when no significant progress is made.
- *Average sampling.* Where several samples of the same size are drawn from the data set, a prediction model is created for each sample, and the outputs of all the models are combined by voting or averaging (more on this in Sect. 10.1).

While discussing data preparation, it is also worthwhile to mention some other aspects of this phase. Some problems require *data normalization* (e. g., scaling some values to a specific range, say, [0, 1]). For example, the age of a car (in number of years) should be interpreted on a different scale than the mileage. In particular, two cars of the same age, but which differ by five miles, can be considered quite similar (assuming that the other variables are the same), whereas two cars of the same mileage, but which differ by five years, are quite different. Data normalization also allows us to express some values as integers, categories, floating point numbers, labels, etc. For instance, we can transform $400 in damage into "damage level" = 0.04, or we can assign damage to 1 of 10 categories, such as category 1 for damage under $500, category 2 for damage between $501 and $1,000, and so forth. As indicated earlier, we can also transform the variable "color" into 20 binary variables, one for each color: "white," "silver," "red," "green," "blue," …, "black."

Another important issue connected with data preparation is that of inaccurate or missing values. Inaccurate values usually arise from typographical errors. Some of these errors can be "discovered" by analyzing the outliers for each variable, but some of them may be difficult to find. Furthermore, in almost any data set, some values are not recorded. For example, the color might be unknown for some cars, or the mileage might be missing. Sometimes missing values are treated as just another variable value (e. g., "white," "silver," "red," …, "black," "unknown"). Another possibility would be to ignore all cases with missing values, but in some data sets we might lose over 90% of the cases by doing this! Yet another way of approaching the problem of missing values is to replace them with the variable's mean value. This might be tempting, but it is very risky, as the data could become biased. Instead, it is safer to observe a relationship between the variable in question and some other variables, and then replace the missing value with an estimated value. For example, we can estimate the mileage on the basis of other variables (such as "year" and "type"): a four-year-old off-lease car should have around 48,000 miles, because car leases typically allow for 12,000 miles per year.

The final aspect of data preparation is connected with time-dependent data. Because all orders, deliveries, and transactions have some sort of a time stamp, most real-world business problems have some time-dependent relationships within their

data sets. Even some relatively "stable" data sets, such as bank customers, change over time. Of course, these types of changes happen at a much slower rate than changes in the stock market, but they do happen. Thus, this additional dimension of time – additional to cases and variables – plays a significant role in most prediction models. This time factor necessitates updates of the prediction model at regular intervals. This can be done online, when new data arrive, or offline, by analyzing the new data and modifying the prediction model. We will return to this issue later, in Sect. 10.3, when we discuss the process of updating a prediction model.

Time dependencies should be recognized and dealt with during the data preparation phase. Usually, time series models assume that the values for some variables are recorded at fixed intervals. For example, we can record the US Gross Domestic Product at the end of each quarter, the Dow Jones Industrial Average at the end of each business day, the temperature at some location every four hours (i. e., six readings a day), and so on. However, if we look at the car distribution example, our time series is far less regular. Although the price prediction is for a particular make/model, there are many subcategories within each make/model category (because of different mileage, color, trim, etc.). Hence, if we find several exact cases from the past, they will not have regular time intervals: For example, a blue Toyota Corolla with 33,000 miles was sold on April 13th, two more were sold in early May, and another was sold in late August – but we have to make prediction for this exact car for mid-October. Because of these interval irregularities, we should relax the precision of some input variables. For instance, color need not be exactly the same (and for some makes/models, color is not a major influencer of price anyway). On top of everything, we are not predicting the value of a variable for the "next" time unit. If we ship a car from California to an auction site in Arizona, we might be interested in a price prediction for next week (as the shipping time would be several days). On the other hand, if we ship the same car to an auction site in New York, then we might be interested in a price prediction for three weeks from today! Needless to say, the volume effect should also be taken into account, as other distribution decisions may influence the actual price of the car!

Another important issue related to time-dependency is the "time horizon" of the historical data. Simply put, we have to make a decision on how far back to look. It seems natural that we should pay more attention to recent data, as "old" data may have lost their significance. For example, using pre-September 11th, 2001 data to predict air traffic for 2002 would not yield good results.

Some people also consider a preliminary (exploratory) analysis to be a part of the data preparation phase, while others consider it a separate stage of the data mining process. In either case, such an analysis is extremely helpful for gaining an understanding of the data. Preliminary data analysis usually includes graphing data for visual inspection (e. g., we can graph the prices for a particular make/model with respect to mileage), and computing some simple statistics such as averages, minimums, maximums, means, standard deviations, and percentiles for each data set (e. g., the prices of a particular make/model at a particular auction site). We can also use "decomposition analysis" to detect trends, seasonality, cycles, and to identify

outliers. Anyway, the main purpose of such an analysis is not to immediately select a prediction model, but rather to get a "feel" for data. This stage is vital, as it can suggest the appropriate prediction method.

5.2 Different Prediction Methods

After the data are prepared, we can begin our search for the right prediction method. The goal is to build a prediction model that will predict the "outcome" of a new case. This outcome might be the price of a used car sent to auction, the classification of a loan application, the assignment of a new customer to the appropriate cluster, and so on. Many prediction methods have been developed over the years that differ from one another in the representation of a solution (e. g., decision tree versus a set of rules), as well as some other differences (e. g., whether they are capable of "explaining" the prediction, the ease with which a solution can be edited). We can group these different prediction methods into a few broad categories:

- Mathematical (e. g., linear regression, statistical methods).
- Distance (e. g., instance-based learning, clustering).
- Logic (e. g., decision tables, decision trees, classification rules).
- Modern heuristic (e. g., neural networks, evolutionary algorithms, fuzzy logic).

The first three categories are covered in this chapter, but the last category, modern heuristic methods, is covered in later chapters. These heuristic methods include fuzzy systems (Chap. 7), neural networks (Chap. 8), genetic programming (Chap. 9), and agent-based systems (Chap. 9). One can argue, of course, that neural networks can be placed in the category of mathematical models, whereas fuzzy systems and genetic programming are in the category of logic models (as they represent classification rules and decision trees, respectively). However, these techniques are of growing importance for building prediction models, and so we have moved them into separate chapters to discuss them in greater depth.

5.2.1 Mathematical Methods

As discussed earlier in this chapter, there are three types of prediction problems: classification, regression, and time series. Classification problems have been the focus of data mining research for the last few decades, and some prediction methods (e. g., distance and logic) were developed explicitly for classification problems. For the time being, however, let us focus on regression and time series problems.

The major difference between regression and time series problems is that the former assumes that the expected output exhibits some explanatory relationship with some other variables. For example, someone's (predicted) salary might be a function of education, experience, industry, and location. In such cases, an explanatory method would be used to find the relationship between these variables

and make a prediction. The goal of time series models, on the other hand, is not to discover or explain the relationships between variables; their goal is purely one of prediction. Neural networks (Chap. 8) are a good example of this: We may not understand the connection weights, the importance of particular variables or their relationship, and yet the neural network model might be producing quite accurate predictions …

Probably the most popular explanatory method is *linear regression* . If the predicted outcome is numeric and all the variables in the prediction model are numeric, then linear regression is the classic choice.[3] In this method, we build a linear expression that uses the values of different variables to produce a predicted value for a "new" variable (i. e., a variable not used in the model). To illustrate this prediction method in more detail, let us consider linear regression for predicting the auction price of a car. In this case, the "new" variable would be the predicted sale price. Note that many variables are *not* numeric, so we have to address this issue first. It is clear that the non-numeric variables "make," "model," and "location" are of key importance, as they determine the basic price range (which is further influenced by the mileage, year, trim, etc.). By building a separate regression model for each make/model at each location, we can eliminate these three non-numeric variables.

Next, we should convert the remaining non-numeric variables into numeric variables. For example, we can take a list of the available colors, sort them from white to black according to some standard order (e. g., how they appear on a spectrum), and assign consecutive natural numbers. Assuming we have 30 different colors, *white* would be 1 and *black* would be 30. Similar assignments can be made for other non-numeric variables. Note that the variables "mileage," "year," and "damage level" are already numeric, so there is no need to covert these.

Because a linear regression model must answer (i. e., produce a value for) questions such as: "What's the price of a Toyota ("make") Camry ("model") at auction site Jacksonville, Florida ("location")?"[4] we need to develop a function:

$$\text{Sale Price} = a + (b \times \text{Mileage}) + (c \times \text{Year}) + (d \times \text{Color}) + \ldots$$

that provides the predicted price for a new case (i. e., a used Toyota Camry) when supplied with the numeric values of the other variables ("mileage," "year," "color," etc.). The main challenge here is to find the values for parameters *a, b, c, d,* etc. that give the prediction model the best possible performance (i. e., that minimize the predictive error). Since we have all the historic data from three million cases, we can extract all cases where "location" = Jacksonville, "make" = Toyota, and "model" = Camry. This subset of cases (say we identified 150 such cases) would constitute the data set available for training the prediction model (some of

[3] Note also that in some situations we would like to predict only one of two values ("yes" or "no," "fraudulent" or "legitimate," "buy" or "sell," etc.). This type of regression is called logistic regression, and a similar methodology is applied (e.g., transformation of variables, building a linear model).

[4] Note that the Jacksonville location would contain many prediction models (for all distinct pairs of make/model).

these cases would also be used for validation and testing; see Sect. 5.3 for more details on this).

To minimize the error on the training set, there are several standard procedures for determining the parameter values. Once these parameters are determined, the prediction model (for all Toyota Camry cars sold at the Jacksonville auction) is ready. For every new case (again, by *new* case we mean a *used* Toyota Camry), we can determine the sale price for the Jacksonville location by inserting the appropriate values for "mileage," "year," "color," etc. into the sale price function.

Note, however, that the training process might not be that simple (this is true for any prediction model, not just linear regression). First of all, some values might be missing (e. g., the mileage was not recorded). In such cases we can:

- Remove the case from consideration and contact the appropriate auction site to recover the mileage value. Once this value is recovered, we can insert the case back into the system for processing. Although this would cause a delay in processing the car, it might prevent us from making a serious prediction error.
- Estimate the mileage on the basis of other variables. For example, if the car was "leased," it might be reasonable to assume that the average mileage allowance is 12,000 miles per year. Thus, a three-year-old car is likely to have 36,000 miles.

Second, because the prediction model has to provide more than just tomorrow's price (as it takes some time to transport the car to Jacksonville, and so we need a predicted price for next week and/or three weeks from today), the training process might be much more complex. The reason for this increased complexity is hidden in the fact that the prediction model's accuracy must be assessed for both shorter and longer time periods. Hence, the process of searching for the best prediction model is more difficult, as it is harder to compare and select the better of two models where one provides better short-term predictions while the other provides better longer-term predictions. This is a typical multi-objective optimization problem (as discussed in Sect. 2.4).

Third, from time to time the linear regression model would process a "rare" car, such as a Dodge Viper or Acura NSX. Note that we assumed a linear regression model for each make/model at each location. This assumption is fine, but the historical data set may only contain 100 Dodge Viper cars with zero occurrences at some locations! How can we build a model for a location where the data set is empty? Well, as usual, there are several ways of dealing with this problem. One way would be to estimate the price on the basis of (1) prices of the same make/model at nearby locations, and (2) prices of similar models at the same location. This approach would require some additional, problem-specific knowledge. Another possibility would be to use an approach based on agent modeling (Chap. 9), which can be used as a data mining technique for "data-less" problems!

The above example serves to underline the simple fact that *the devil is in the detail.* This is true for any prediction method, because developing a prediction model for a real-world problem usually involves the resolution of many issues ranging from incomplete information to insufficient data. Something else to consider is that regression might be far more complicated than our simple example. Note that the prediction model above has one powerful disadvantage: it is *linear*! Real-world

data often display nonlinear dependencies that we would like to capture (recall the nonlinear transportation model in Sect. 2.5). Of course, a linear regression model would find the best possible line, but the line may not fit very well.

One approach to this problem is to replace the line with a curve, which can be done by transforming the variables (by multiplying some of them together, squaring or cubing them, or taking their square root). After completing these transformations, we can then determine the new parameters (i. e., *a*, *b*, *c*, *d*, etc.) of the prediction model (although this new model is more complex and we are now talking about *nonlinear* regression). It is possible to experiment with a wide variety of transformations, and if they do not provide a meaningful contribution to the prediction model, then their parameters will stay close to zero. The difficulty, however, is that the number of possible transformations might be too prohibitive (i. e., the number of possible parameters to explore might be too high, and any training would be infeasible). Moreover, with complex transformations we should guard against overfitting,[5] as the use of complex transformations guarantees high precision on the training set that may not carry over to new predictions.

Now let us turn our attention to time series problems. As mentioned earlier, the only purpose of a time series model is to predict future values; the relationships between the variables are of no interest. The problem might be expressed as follows:

Given v[1], v[2], …, v[t], predict the values of v[t+1], v[t+2], …, v[t+k]

where *t* is the present time interval, *t–1* is the previous time interval, *t+1* is the next time interval, and so on. If we are only predicting the next interval (*t+1*), then a time series model is concerned with a function *F* such that:

$$v[t+1] = F(v[1], v[2], \ldots, v[t])$$

Note that the above function may include some other variables, and not just the values of variable *v* from earlier time intervals. In such cases, we talk about *composite forecasting models*, which consist of past time series values, past variables, and past errors.

Many statistical time series models have been proposed during the last few decades, including exponential smoothing models, autoregressive/integrated/moving average models, transfer function models, state space models, and others. Each model is based on some assumptions, and involves a few (at least one) parameters that must be tuned on the basis of historical data.

Now let us consider the category of prediction methods that are collectively known as "exponential smoothing." These methods generalize the *moving average method*, where the mean of past *k* cases is used as a prediction. All exponential smoothing methods assign weights to past cases in such a way that recent cases are given more weight than the older cases (as the more recent cases usually provide better future direction than the less recent ones). Hence, it is reasonable to develop a weighting scheme that assigns smaller weights to older cases. Such a weighting

[5] *Overfitting* occurs when a model tunes itself during the training stage to such an extent that all predictions on the training data set are perfect.

scheme also requires at least one parameter *a*. For example, a prediction for the time *t+1* is calculated as:

$$\text{Prediction}(t+1) = (a \times \text{Actual}(t)) + ((1-a) \times \text{Prediction}(t))$$

which simply means that the prediction for the next (future) case is calculated as a total of two values: the actual last case (*Actual(t)*) with parameter *a* and the last prediction (*Prediction(t)* with the weight *1–a*). Note that parameter *a* provides the significance of the last case in making the prediction; in particular, if $a = 1$, then the prediction would always report the last actual value as a new prediction. It is easy to generalize this method to include more past cases:

$$\begin{aligned}\text{Prediction}(t+1) = {} & (a \times \text{Actual}(t)) + (a \times (1-a) \times \text{Actual}(t-1)) \\ & + (a \times (1-a)^2 \times \text{Actual}(t-2)) + (a \times (1-a)^3 \times \text{Actual}(t-3)) + \ldots \\ & + (a \times (1-a)^{t-1} \times \text{Actual}(1)) + ((1-a)^t \times \text{Prediction}(1))\end{aligned}$$

so *Prediction*(*t+1*) represents a weighted moving average of all past observations. Note again, that different values of parameter *a* would result in a different distribution of weights. Also, it was assumed that the prediction horizon was just one period away (*t+1*). For longer-term predictions, it is often assumed that the function is flat:

$$\text{Prediction}(t+1) = \text{Prediction}(t+2) = \text{Prediction}(t+3) = \ldots$$

as exponential smoothing works best for data that have no trend or seasonality. However, since some form of trend or seasonality exists in most data sets, *decomposition* methods can be used to identify the separate components of the underlying trend-cycle and seasonal factors. The trend-cycle (which is sometimes separated into trend and cyclical components) represents long-term changes in the time series values, whereas seasonal factors relate to periodic fluctuations of constant length caused by phenomena such as temperature, rainfall, holidays, etc.

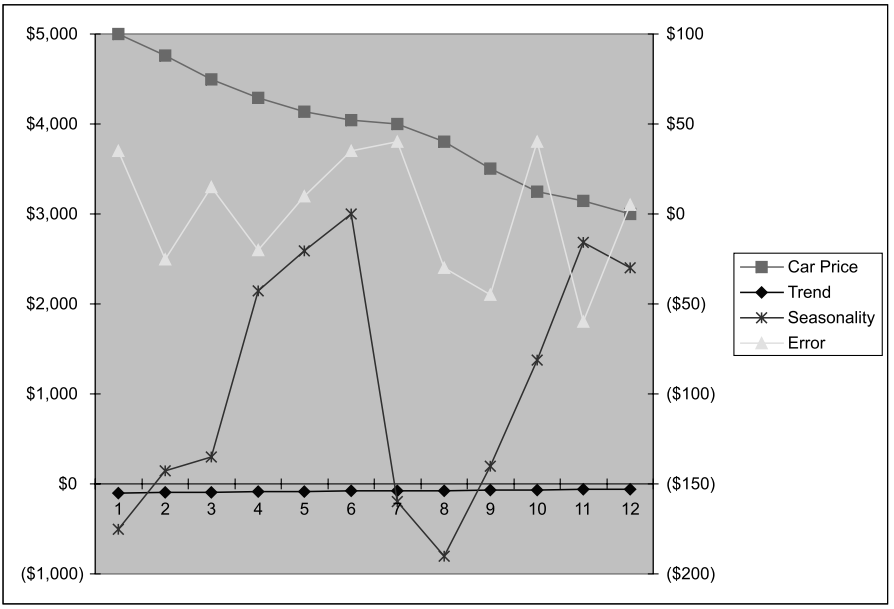
Although there are several approaches to decomposing a time series problem into separate components, the basic concept is based on experience: First the trend-cycle is removed, then the seasonal components are addressed. Any remaining error is attributed to randomness; thus:

$$\text{Data} = \text{trend-cycle} + \text{seasonality factors} + \text{error}$$

Note that the relationship between the data and trend-cycle, seasonality factors, and error need not be linear (additive, as above); in general, decomposition methods search for a function *D* that would "explain" a data point at any time *t*:

$$\text{Data}(t) = D(\text{trend-cycle}(t), \text{seasonality factors}(t), \text{error}(t))$$

The figure below illustrates what such a decomposition of data might look like for the car distribution example:



First of all, note that the "Car Price" (Data) corresponds to the left y-axis, while the "Trend", "Seasonality" and "Error" correspond to the right y-axis, and the x-axis represents the month. In general, the "Trend" is a continued decrease of the "Car Price," while "Seasonality" can have a negative effect or no effect at all. The "Error" can be positive or negative. Let us take June (month "6" in the figure above) as an example: The "Car Price" is $4,045, the "Trend" during June is a decrease of $152, the "Seasonality" effect is $0, and the "Error" is $35. If we add up all these numbers then we get a "Car Price" of $3,928 for the beginning of July.

Going back to exponential smoothing, the relationship between the past and future is linear, but this might not be appropriate for many real-world applications of time series. Linear models cannot capture some features that commonly occur in actual data, such as asymmetric cycles (which are data patterns in which the period of repeating cycles is not fixed, and the average number of data on the up-cycle is different than the average number of data on the down-cycle) and occasional outliers. Although linear methods often deal with nonlinear time series by logarithmic or power transformations of data, these techniques do not account for asymmetric cycles and outliers.

Some nonlinear methods assume that asymmetric cycles are caused by distinct underlying phases of the time series, and that a transition period (either smooth or abrupt) exists between these phases. The individual phases are usually given a linear functional form, and the transition period (if smooth) is modeled as an exponential or logarithmic function. Some other methods are used to deal with time series that display variable variance of residuals (error values). In these methods,

the variance of error values is modeled as a quadratic function of past variance values and past error values.

Although all these linear and nonlinear methods are capable of characterizing the variables found in actual data, they also assume that the underlying process of data generation is constant. This assumption is often invalid for actual time series data, as changing environmental conditions may cause the underlying data generating process to change. For all prediction methods, human judgment is required to first select an appropriate method, and then set the appropriate parameter values for the model's parameters. In the event that the underlying data generating process changes, the time series data must be revaluated and the parameter values readjusted (in extreme cases, a new model might be required). We will address the issue of adaptability in Sect. 10.3, as well as a few other subsections in Part II of this book.

5.2.2 Distance Methods

Another method for building prediction models is based on the concept of "distance between cases." Any two cases in a data set can be compared for similarity, and this similarity measure (called "distance") is assigned some value: the more similar the cases, the smaller the value. Using a distance measure within a data set would allow us to compare a new case with the most "similar" existing case. The outcome of the most similar case (e. g., the loan was repaid, the transaction was fraudulent) would be the prediction for the new case. Going back to our example of the Toyota Camry at the Jacksonville auction site, we may search our database of three million cases for the most similar Toyota Camry sold in Jacksonville and use its sale price as our prediction. Ideally, the existing case would be recent and have the same mileage, color, trim, etc. as the new case. Hence, instead of building a function where the variable values (magnified by some weights) determine the outcome, we just keep the past cases.

The essential aspect of this approach is creating a similarity measure between cases, because the probability of finding an identical case is very low. Hence, we have to base our decisions on similarities, which is far from trivial: For instance, is a silver Toyota Camry with 33,000 miles "more similar" to a white Toyota Camry with 34,100 miles, or to a silver Toyota Camry with 36,000 miles? Or, is the difference in "similarity" between "silver" and "white" the same as between "red" and "yellow"? To answer such questions, it is necessary to define some *distance* between cases (again, the shorter the distance, the greater the similarity).

One of the most popular distance-based prediction methods is *k nearest neighbor*, where k nearest neighbors (i. e., k most similar cases) of a new case are determined. Clearly, if $k = 1$ (i. e., we find only one neighbor), the outcome of this single neighbor is the prediction for the new case. If $k > 1$, then a voting mechanism is used (classification problems) or the average value of the k answers is calculated (regression problems).

Note, however, that the most important step of the k nearest neighbor method is calculating the distance between cases – this is crucial for getting high-quality

results. There are many ways of defining a distance function, but experimentation is often the best way. In any case, careful data preparation is always the first step (it is likely that the data will be normalized to equalize the scale for computing distances and/or some weighting will be applied where different variables get different weights). Note that calculating the distance is trivial when there is only one numeric variable, (e. g., 5.7 – 3.8 = 1.9). With several numerical variables, a Euclidean distance[6] can be used, provided that the variables are normalized and of equal importance (otherwise a weighting must be applied).

The largest problem, however, is with nominal variables. Given our earlier question of whether "the difference in similarity between 'silver' and 'white' is the same as between 'red' and 'yellow'?" we can assume that different colors are just different (resulting in a distance of 1), or we can introduce a more sophisticated matrix that would assign a numeric measure for each color (e. g., so that the difference between "light blue" and "dark blue" is smaller than the difference between "blue" and "red"). These are the two standard approaches for evaluating differences between the values of nominal variables.

Another issue to consider is that of missing values. A standard approach is to assume that the distance between an existing value and a missing value is as large as possible. Hence, for nominal values, the distance is assigned a normalized value of 1 (all distances are between 0 and 1), and for numeric variables the distance is assigned the largest possible normalized value between 0 and 1. For example, if an existing value is 0.27 and the other value is missing, then the distance is 0.73; if the existing value is 0.73 and the other value is missing, then the distance is also 0.73.

Yet another issue is the number of stored cases. A distance-based method might be too time consuming for large data sets, because the whole data set must be searched to evaluate each new case. With larger values of parameter *k*, the computation time increases significantly. For efficiency reasons, it would be beneficial to reduce the number of stored cases. By selecting a subset of "representative cases," the process of finding the closest neighbor (or neighbors) might be more efficient. And to make the representative cases as "representative" as possible (i. e., as good as possible), a new set of representative cases can be selected from the current representative cases and all misclassified cases that produced a prediction error larger than some threshold. In other words, the current representative and misclassified cases could constitute an input for some reclassification method (e. g., decision trees), which would be responsible for creating a better set of representative cases.

Also, some clustering methods can be used to group the cases into meaningful categories. A new case would then be assigned to an existing category and the predicted value would be drawn from the cases present in that category (again, by voting for classification, or averaging for regression). Note that it is not necessary to store all the cases per category; again, we can select some representative cases instead. A few clustering techniques might be considered for this task

[6] *Euclidean distance* is defined as the length of a line segment between two points in an *n*-dimensional space. In particular, the distance *d* between two points (x_1, y_1) and (x_2, y_2) in a 2-dimensional space is determined by the following function: $d^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$.

(e. g., *k*-means algorithm, incremental clustering, or statistical clustering based on a mixture model), which we will discuss later in the text.

5.2.3 Logic Methods

A *decision table* (also known as a *lookup table*) is the simplest logic-based method for prediction, and there are many such tables published for estimating the price of a used car sold at auction (e. g., *Black Book, Kelley Blue Book, Manheim Market Report*). In these tables, we can locate the appropriate make/model/year/body style, get a basic price, and adjust this price for additional variables such as mileage, color, trim, damage level, etc. However, not all variables are included (e. g., for some makes/models, *color* might not be included).

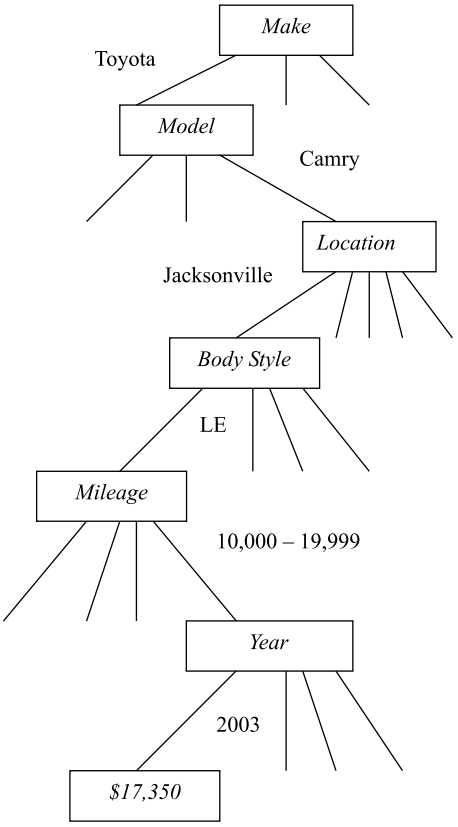
The most widely used logic method, on the other hand, is the *decision tree*. Because the structure of a decision tree is relatively easy to follow and understand (especially for smaller trees), its popularity is widespread. To make a prediction for a new case, the root of a tree is examined, a test is performed,[7] and, depending on the result of the test, the case moves down the appropriate branch. The process continues until a terminal node (also known as a "leaf") is reached, and the value of this terminal node is the predicted outcome.

Although decision trees are used for all types of predictions problems, they are especially popular for classification problems. If the test involves a nominal variable, the number of branches corresponds to the number of possible values that variable can take (i. e., there is one branch for each possible value). If the test involves a numeric variable, there are usually two branches, as the test determines whether the value is "greater than" or "less than" (possibly also "equal to" for integer numbers) some predefined fixed value.[8] In the case of missing values, an additional branch is assigned or some other heuristic is used (e. g., selection of the most popular branch or selection of a few branches).

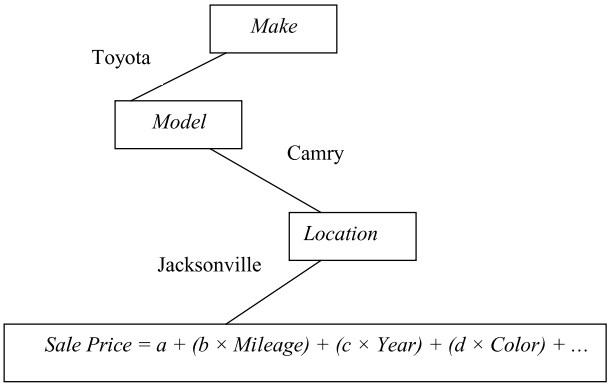
We can easily picture a decision tree for used car prices. At the root of the tree a decision is made on "make": if there are 30 different makes, then the root node would have 30 branches. On the second level of the tree, a decision is made on "model," then the third level provides branches for "location." Further down, we may have nodes that test a new case for "body style" and "mileage" and refer it to the appropriate branch. For example, a test on "mileage" might involve the selection of an appropriate category (e. g., "0 to 9,999 miles," "10,000 to 19,999 miles," and so on). The following illustrates the branch of a simplified decision tree:

[7] By "test" we mean that a node compares a value of a variable with some constant. However, it is possible to include more sophisticated tests, where more variables and/or additional functions are involved.

[8] It is also possible to have a decision tree with more than two branches for a numeric variable, where a range of values is assigned to each branch.



Naturally, there are better and more sophisticated ways to use decision trees for numeric prediction. It might not be practical to represent every value (or range of values) as a separate branch in a decision tree, as the size of the tree might be too large. Instead of keeping a single, numeric value at each terminal node (as illustrated above), it might be easier to keep a model (e. g., a linear regression model) that predicts a value for all cases that reach this terminal node. Such a tree could be used to answer our question from Sect. 5.2.1: "What's the price of a Toyota ("make") Camry ("model") at auction site Jacksonville ("location")?" The variables "make," "model," and "location" are used for building the tree (and later for branch determination when processing a new case), whereas mileage, year, color, etc. are used as variables in the linear regression model at each terminal node, as illustrated below:



As before, the predicted sale price might be adjusted further to take into account the time factor, as it would take some time to transport the car to Jacksonville. Note that the number of parameters (*a*, *b*, *c*, *d*, etc.) and their values might be different at each terminal node.

To achieve better prediction accuracy, it might be worthwhile to build a linear regression model for each node of the tree (rather than just for the terminal nodes).[9] Note, however, that the root node would now have a function that involves all the variables:[10]

$$\text{Sale Price} = a + (b \times \text{Make}) + (c \times \text{Model}) + (d \times \text{Location}) + (e \times \text{Mileage}) + (f \times \text{Year}) + (g \times \text{Color}) + \ldots$$

For second-level nodes, the linear function will not include the variable "make," because the appropriate branch of the decision tree has already been selected. Thus, the linear function would be:

$$\text{Sale Price} = a + (b \times \text{Model}) + (c \times \text{Location}) + (d \times \text{Mileage}) + (e \times \text{Year}) + (f \times \text{Color}) + \ldots$$

For third-level nodes (where the decision for make and model has already been made), the function would be:

$$\text{Sale Price} = a + (b \times \text{Location}) + (c \times \text{Mileage}) + (d \times \text{Year}) + (e \times \text{Color}) + \ldots$$

and so on. (Note, however, that the parameters *a*, *b*, *c*, etc. are different in all these functions.) In our earlier diagram, the terminal node was placed on the fourth level with a linear function of:

$$\text{Sale Price} = a + (b \times \text{Mileage}) + (c \times \text{Year}) + (d \times \text{Color}) + \ldots$$

[9] Experimental evidence shows that prediction accuracy can be increased by combining several prediction models together (we will discuss this in Sect. 10.1).

[10] This approach usually involves nominal attributes as well (e.g., make, model, location) as all the variables are represented on different levels of the decision tree. Such nominal variables are transformed into binary variables and treated as numeric.